

Specyfikacja Wymagań Projektowych: System Rezerwacji Hoteli

1. Cel i krótki opis projektu Celem projektu jest stworzenie prostej, obiektowej aplikacji (działającej w konsoli lub weryfikowalnej wyłącznie za pomocą testów jednostkowych) symulującej logikę systemu rezerwacji hoteli. Projekt ma na celu praktyczne zastosowanie zasad czystego kodu oraz metodyki TDD.

2. Główne encje systemu (Struktura klas)

- **Hotel / SystemRezerwacji:** Główna klasa zarządzająca listą pokoi oraz przetwarzająca zlecenia rezerwacji.
- **Room (Pokój):** Obiekt przechowujący informacje o numerze pokoju, jego standardzie (np. enum RoomType) oraz cenie za noc.
- **Guest (Klient):** Obiekt reprezentujący osobę dokonującą rezerwacji (np. imię, nazwisko, ID klienta).
- **Reservation (Rezerwacja):** Obiekt łączący Klienta, Pokój oraz przedział czasowy (np. LocalDate od-do), przechowujący również status rezerwacji (np. POTWIERDZONA, ANULOWANA).

3. Wymagania Funkcjonalne (Logika biznesowa)

- System umożliwia dodawanie nowych obiektów typu Room do puli dostępnych pokoi.
- System pozwala na wyszukanie wolnych pokoi w zadanym przedziale czasowym (LocalDate start, LocalDate end).
- System pozwala na utworzenie obiektu Reservation dla danego Guest i przypisanie do niego dostępnego Room.
- System posiada funkcję obliczania całkowitego kosztu rezerwacji (cena pokoju mnożona przez liczbę dni).
- System pozwala na zmianę statusu rezerwacji na "Anulowana", co zwalnia przypisany pokój w danym terminie.
- System rzuca odpowiedni wyjątek (np. RoomUnavailableException), gdy użytkownik próbuje zarezerwować pokój, który jest już zajęty.

4. Wymagania Niefunkcjonalne i Techniczne

- **Technologia:** Java.
- **Architektura:** Brak interfejsu graficznego; komunikacja odbywa się poprzez wywoływanie metod na obiektach lub proste menu konsolowe.

- **Metodyka:** Rozwój oprogramowania oparty na TDD (Test-Driven Development) – najpierw pisane są testy (np. w JUnit), a następnie kod implementacyjny.
- **Jakość kodu:** Kod musi być zgodny z zasadami "Clean Code" i podzielony na sensowne pakiety/klasy.
- **Kontrola wersji i Organizacja:** Praca na repozytorium z wykorzystaniem podejścia Feature Branch oraz przeprowadzanie Code Review dla każdego Pull Requesta.
- **Automatyzacja:** Projekt wykorzystuje system Continuous Integration (CI), np. GitHub Actions, do automatycznego uruchamiania testów po każdym commicie/PR.

5. Główne zasady i reguły walidacyjne

- Daty zakończenia rezerwacji muszą być zawsze późniejsze niż daty rozpoczęcia.
- Niedozwolony jest "overbooking" – dwa obiekty Reservation o statusie "Aktywna" nie mogą współdzielić tego samego obiektu Room w nakładających się przedziałach czasowych.